

NAME-POOJA MAHTO	BATCH-CH3
PRN-202501050057	COARSE-EDS

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the groupby and mean() operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.



Sample Test Cases

Test case 1

```
1 2 3 4 5 6 7 8 9 10
```

```
Mean of even and odd numbers:↵
```

```
Odd.....5.0↵
```

```
Even....6.0↵
```

```
dtype: float64↵
```

Test case 2

```
4 4 4 6 6 2 2 2 10 12 14 16
```

Mean of even and odd numbers:

Even ... 6.833333

dtype: float64

Test case 3

1 5 7 9 11 13 15 17 111 21 25

Mean of even and odd numbers:

Odd ... 21.363636

dtype: float64

CODE:

```
import pandas as pd
```

```
# Take inputs from the user to create a list of numbers
```

```
numbers = list(map(int, input().split()))
```

```
# Create a Pandas series from the list of numbers
```

```
series = pd.Series(numbers)
```

```
# Grouping by even and odd numbers and calculating the mean
```

```
grouped = series.groupby(series % 2 == 0).mean()
```

```
# Display the mean of even and odd numbers with labels
```

```
grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
```

```
print("Mean of even and odd numbers:")
```

```
print(grouped)
```

Average time
0.007 s
6.67 ms

Maximum time
0.010 s
10.00 ms

✓ 3 out of 3 shown test case(s) passed

✓ 3 out of 3 hidden test case(s) passed

✓ Test case 1 10 ms Debug

Expected output	Actual output
1 2 3 4 5 6 7 8 9 10	1 2 3 4 5 6 7 8 9 10
Mean of even and odd numbers:	Mean of even and odd numbers:
Odd ... 5.0	Odd ... 5.0
Even ... 6.0	Even ... 6.0
dtype: float64	dtype: float64

✓ Test case 2 6 ms Debug

Expected output	Actual output
4 4 4 6 6 2 2 2 10 12 14 16	4 4 4 6 6 2 2 2 10 12 14 16
Mean of even and odd numbers:	Mean of even and odd numbers:
Even ... 6.833333	Even ... 6.833333
dtype: float64	dtype: float64

✓ Test case 3 7 ms Debug

Expected output	Actual output
-----------------	---------------

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the **Age** column.
- Display the DataFrame after deleting the column.



Sample Test Cases

Test case 1

```
Original DataFrame:
  Name Age
0 Alice 25
1 Bob   30
2 Charlie 35
New name: Susan
New age: 29
After adding a row:
  Name Age
0 Alice 25
1 Bob   30
2 Charlie 35
3 Susan 29
Index of row to modify: 0
New age: 27
After modifying a row:
  Name Age
0 Alice 27
1 Bob   30
2 Charlie 35
3 Susan 29
Index of row to delete: 3
After deleting a row:
  Name Age
0 Alice 27
1 Bob   30
2 Charlie 35
Enter genders separated by space: F M M
After adding a new column:
  Name Age Gender
0 Alice 27 F
1 Bob 30 M
2 Charlie 35 M
After modifying a column:
  Name Age Gender
0 ALICE 27 F
1 BOB 30 M
```

```

2 ···CHARLIE ···35 ······M↵
After deleting a column:↵
·····Name ·Gender↵
0 ····ALICE ······F↵
1 ·····BOB ······M↵
2 ···CHARLIE ······M↵

```

CODE:

```

import pandas as pd

# Provided dictionary of lists
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
}

# Convert the dictionary to a DataFrame
df = pd.DataFrame(data)

# Display the original DataFrame
print("Original DataFrame:")
print(df)

# Adding a new row
new_name=input("New name: ")
new_age=int(input("New age: "))
df.loc[len(df)]=[new_name,new_age]

# Display the DataFrame after adding a new row
print("After adding a row:\n",df)

# Modifying a row
row_to_modify= int(input("Index of row to modify: "))
new_age_value= int(input("New age: "))
df.at[row_to_modify,"Age"]=new_age_value

# Display the DataFrame after modifying a row
print("After modifying a row:")
print(df)

# Deleting a row
row_to_delete=int(input("Index of row to delete: "))
df=df.drop(index=row_to_delete).reset_index(drop=True)

```

Average time

0.050 s

50.00 ms



Maximum time

0.051 s

51.00 ms



✓ 1 out of 1 shown test case(s) passed

✓ 1 out of 1 hidden test case(s) passed

✓ Test case 1 51 ms

Debug



Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note:

Refer to the displayed test cases for better understanding.



Sample Test Cases

Test case 1

studentdata.txt

First five rows:

···Name···Age···Grade

0··John···25·····A

1··Alin···22·····B

2··Emma···24·····A

3··Mike···23·····C

4··Sony···21·····B

Average age: 23.29

Students with a grade up to B

···Name···Age···Grade

0··John···25·····A

1··Alin···22·····B

2··Emma···24·····A

4··Sony···21·····B

5··Amia···26·····A

CODE:

```
import pandas as pd
```

```
# Read the text file into a DataFrame
```

```
file = input()
```

```
data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])
```

```
# write your code here..
```

```
print("First five rows:")
print(data.head())
```

```
avg_age = round(data["Age"].mean(), 2)
print("Average age:", avg_age)
```

```
filtered_data = data[data["Grade"].isin(["A", "B"])]
```

```
print("Students with a grade up to B")
print(filtered_data)
```

Average time

0.018 s

18.00 ms



Maximum time

0.021 s

21.00 ms



✓ 1 out of 1 shown test case(s) passed

✓ 1 out of 1 hidden test case(s) passed

✓ Test case 1 21 ms

Debug



Expected output

Actual output

studentdata.txt

studentdata.txt

First five rows:

First five rows:

...Name...Age...Grade

...Name...Age...Grade

0..John...25....A

0..John...25....A

1..Alin...22....B

1..Alin...22....B

2..Emma...24....A

2..Emma...24....A

3..Mike...23....C

3..Mike...23....C

4..Sony...21....B

4..Sony...21....B

Average age: 23.29

Average age: 23.29

Students with a grade up
to B

Students with a grade up
to B

...Name...Age...Grade

...Name...Age...Grade

0..John...25....A

0..John...25....A

1..Alin...22....B

1..Alin...22....B

2..Emma...24....A

2..Emma...24....A

4..Sony...21....B

4..Sony...21....B

5..Amia...26....A

5..Amia...26....A

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.



Sample Test Cases

Test case 1

sales_data.csv

Best month: 2025-01

Total sales: \$1210.00

CODE:

```
import pandas as pd

# Prompt the user for the file name
file_name = input()

# Load the data
df = pd.read_csv(file_name)
df['Sales'] = df['Quantity'] * df['Price']

df['Month'] = pd.to_datetime(df['Date']).dt.to_period('M')
```



```
monthly_sales = df.groupby('Month')['Sales'].sum()

# Find the month with the highest total sales
best_month = monthly_sales.idxmax()
highest_sales = monthly_sales.max()

print(f'Best month: {best_month}')
print(f'Total sales: ${highest_sales:.2f}')
```

Average time
0.010 s
9.67 ms

Maximum time
0.013 s
13.00 ms

✓ 1 out of 1 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 13 ms

Debug

Expected output

Actual output

sales_data.csv

sales_data.csv

Best month: 2025-01

Best month: 2025-01

Total sales: \$1210.00

Total sales: \$1210.00

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

**Sample Test Cases****Test case 1**

sales_data.csv

Best selling product: Product A

Total quantity sold: 23

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York

2025-01-01,Product B,3,15,Los Angeles

2025-01-02,Product A,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product B,2,15,Chicago

2025-01-03,Product A,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product B,5,15,Los Angeles

2025-01-05,Product A,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

**Sample Test Cases****Test case 1**

sales_data.csv

Best selling product: Product A

Total quantity sold: 23

CODE:

```
import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data
```

```
df = pd.read_csv(file_name)
```

```
df['Sales'] = df['Quantity'] * df['Price']
```

```
df['Month'] = pd.to_datetime(df['Date']).dt.to_period('M')
```

```
monthly_sales = df.groupby('Month')['Sales'].sum()

# Find the month with the highest total sales
best_month = monthly_sales.idxmax()
highest_sales = monthly_sales.max()

print(f'Best month: {best_month}')
print(f'Total sales: ${highest_sales:.2f}')
```

Average time
0.011 s
11.00 ms

Maximum time
0.014 s
14.00 ms

✓ 1 out of 1 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 14 ms

Debug

Expected output

Actual output

sales_data.csv

sales_data.csv

Best month: 2025-01

Best month: 2025-01

Total sales: \$1210.00

Total sales: \$1210.00

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

**Sample Test Cases****Test case 1**

sales_data.csv

Best ·selling ·product: ·Product ·A↵

Total ·quantity ·sold: ·23↵

CODE:

```
import pandas as pd
```

```
# Prompt the user for the file name
file_name = input()
```

```
# Load the data
df = pd.read_csv(file_name)
```

```
# Find the product with the highest total quantity sold
product_totals = df.groupby('Product')['Quantity'].sum()
best_product = product_totals.idxmax()
highest_quantity = product_totals.max()
```

```
# Display the result
print(f"Best selling product: {best_product}")
print(f"Total quantity sold: {highest_quantity}")
```

Average time

0.008 s

7.67 ms

Maximum time

0.009 s

9.00 ms

✓ 1 out of 1 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1

9 ms

Debug

Expected output	Actual output
sales_data.csv	sales_data.csv
Best-selling product: Product A	Best-selling product: Product A
Total quantity sold: 23	Total quantity sold: 23

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.



Sample Test Cases

Test case 1

sales_data.csv

City sold the most products: Los Angeles

CODE:

```
import pandas as pd
```

```
# Prompt the user for the file name
file_name = input()
```

```
# Load the data
df = pd.read_csv(file_name)
```

```
city = df.groupby('City')['Quantity'].sum()
best_city = city.idxmax()
```

```
# Display the result
print(f"City sold the most products: {best_city}")
```

Average time

0.008 s

8.00 ms



Maximum time

0.009 s

9.00 ms



✓ 1 out of 1 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 9 ms Debug ⋮ 📄 ^

Expected output	Actual output
sales_data.csv	sales_data.csv
City sold the most products: Los Angeles	City sold the most products: Los Angeles

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Explanation:

Transactions:

- **2025-01-01:** Product A, Product B
- **2025-01-02:** Product A, Product C
- **2025-01-03:** Product B, Product A
- **2025-01-04:** Product C, Product B
- **2025-01-05:** Product A, Product C

Now, let's count how often the pairs of products appear together:

- **Product A and Product B:** Appear in transactions on 2025-01-01 and 2025-01-03.
- **Product A and Product C:** Appear in transactions on 2025-01-02 and 2025-01-05.
- **Product B and Product C:** Appears in transactions on 2025-01-04.

Most Frequent Product Combinations:

- **Product A and Product B** (2 times)
- **Product A and Product C** (2 times)

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.



Sample Test Cases

Test case 1

sales_data.csv

Product ·A·and·Product ·B: ·2·times↵

Product ·A·and·Product ·C: ·2·times↵

CODE:

```
import pandas as pd
from itertools import combinations
from collections import Counter

# Prompt user to input the file name
file_name = input()

# Read data from the specified CSV file
df = pd.read_csv(file_name)

# write the code
daily_transactions = df.groupby('Date')['Product'].apply(list)
pair_counts = Counter()

# 2. Iterate through each date's transaction
for products in daily_transactions:
    products = sorted(products)

# Generate all possible pairs (Combinations of size 2)
# If a date has fewer than 2 products, this produces nothing (correctly)
pairs = list(combinations(products, 2))

# Update the counter with these pairs
pair_counts.update(pairs)

# 3. Find the maximum frequency count and print all pairs that match this maximum
frequency
if pair_counts:
    max_count = max(pair_counts.values())

    for pair, count in pair_counts.items():
        if count == max_count:
            print(f"{pair[0]} and {pair[1]}: {count} times")
        else:
            print("No product pairs found.")
```


Average time
0.007 s
7.00 ms

Maximum time
0.008 s
8.00 ms

✓ 1 out of 1 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 8 ms

Debug

Expected output

Actual output

sales_data.csv

sales_data.csv

Product · A · and · Product
· B : · 2 · times

Product · A · and · Product
· B : · 2 · times

Product · A · and · Product
· C : · 2 · times

Product · A · and · Product
· C : · 2 · times

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using .info()).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using .describe().
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (mode()).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

CODE:

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
data = pd.read_csv("Titanic-Dataset.csv")
```

```
# 1. Display the first 5 rows of the dataset
print(data.head())
```

```
# 2. Display the last 5 rows of the dataset
```

```
print(data.tail())
```

```
# 3. Get the shape of the dataset
```

```
print(data.shape)
```

```
# 4. Get a summary of the dataset (info)
```

```
print(data.info())
```

```
# 5. Get basic statistics of the dataset
```

```
print(data.describe())
```

```
# 6. Check for missing values
```

```
print(data.isnull().sum())
```

```
# 7. Fill missing values in the 'Age' column with the median age
```

```
median_age=data['Age'].median()
```

```
data['Age'].fillna(median_age,inplace=True)
```

```
mode_embarked=data['Embarked'].mode()[0]
```

```
data['Embarked'].fillna(mode_embarked,inplace=True)
```

```
# 9. Drop the 'Cabin' column due to many missing values
```

```
data.drop('Cabin',axis=1,inplace=True)
```

```
# 10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'
```

```
data['FamilySize']=data['SibSp'] + data['Parch']
```

Average time
0.065 s
65.00 ms

Maximum time
0.065 s
65.00 ms

1 out of 1 shown test case(s) passed

Test case 165 ms

Debug

Expected output

Actual output

PassengerIdSurvivedPclassFareCabinEmbarked

01037.2500NaN
S

121171.2833C85
C

23137.9250NaN
S

341153.1000C123
S

45038.0500NaN
S

[5 rows x 12 columns]

PassengerIdSurvivedPclassFareCabinEmbarked

8868870213.00NaN
S

8878881130.00B42
S

8888890323.45NaN
S

8898901130.00C148
C

890891037.75NaN
Q

PassengerIdSurvivedPclassFareCabinEmbarked

01037.2500NaN
S

121171.2833C85
C

23137.9250NaN
S

341153.1000C123
S

45038.0500NaN
S

[5 rows x 12 columns]

PassengerIdSurvivedPclassFareCabinEmbarked

8868870213.00NaN
S

8878881130.00B42
S

8888890323.45NaN
S

8898901130.00C148
C

890891037.75NaN
Q

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

CODE:

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']

# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
data['IsAlone'] = (data['FamilySize'] == 0).astype(int)

# 2. Convert 'Sex' to numeric (male: 0, female: 1)
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

# 3. One-hot encode the 'Embarked' column
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 4. Get the mean age of passengers
print(data['Age'].mean())

# 5. Get the median fare of passengers
print(data['Fare'].median())
print(data['Pclass'].value_counts())

# 7. Get the number of passengers by gender
print(data['Sex'].value_counts())

# 8. Get the number of passengers by survival status
print(data['Survived'].value_counts())

# 9. Calculate the survival rate
print(data['Survived'].mean())

# 10. Calculate the survival rate by gender
print(data.groupby('Sex')['Survived'].mean())
```

Average time

0.015 s

15.00 ms



Maximum time

0.015 s

15.00 ms



✓ 1 out of 1 shown test case(s) passed

✓ Test case 1 15 ms

Debug



Expected output

Actual output

29.69911764705882

29.69911764705882

14.4542

14.4542

3...491

3...491

1...216

1...216

2...184

2...184

Name: Pclass, dtype: int64

Name: Pclass, dtype: int64

0...577

0...577

1...314

1...314

Name: Sex, dtype: int64

Name: Sex, dtype: int64

0...549

0...549

1...342

1...342

Name: Survived, dtype: int64

Name: Survived, dtype: int64

0.3838383838383838

0.3838383838383838

Sex

Sex

0...0.188908

0...0.188908

1...0.742038

1...0.742038

Name: Survived, dtype: float64

Name: Survived, dtype: float64

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

CODE:

```
import pandas as pd
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

```
# 1. Calculate the survival rate by class
```

```
print(data.groupby('Pclass')['Survived'].mean())
print(data.groupby('Embarked_S')['Survived'].mean())
print(data.groupby('FamilySize')['Survived'].mean())
```

```
# 4. Calculate the survival rate by being alone
```

```
print(data.groupby('IsAlone')['Survived'].mean())
```

```
# 5. Get the average fare by class
```

```
print(data.groupby('Pclass')['Fare'].mean())
```

```
# 6. Get the average age by class
```

```
print(data.groupby('Pclass')['Age'].mean())
```

```
# 7. Get the average age by survival status
```

```
print(data.groupby('Survived')['Age'].mean())
```

```
# 8. Get the average fare by survival status
```

```
print(data.groupby('Survived')['Fare'].mean())
```

```
# 9. Get the number of survivors by class
```

```
survivors = data[data['Survived'] == 1]
print(survivors['Pclass'].value_counts())
```

```
# 10. Get the number of non-survivors by class
non_survivors = data[data['Survived'] == 0]
print(non_survivors['Pclass'].value_counts())
```

Average time	Maximum time
0.021 s	0.021 s
21.00 ms	21.00 ms

✓ 1 out of 1 shown test case(s) passed

✓ Test case 1	21 ms	Debug			^
Expected output	Actual output				
Pclass	Pclass				
1...0.629630	1...0.629630				
2...0.472826	2...0.472826				
3...0.242363	3...0.242363				
Name: Survived, dtype: float64	Name: Survived, dtype: float64				
Embarked_S	Embarked_S				
0...0.506073	0...0.506073				
1...0.336957	1...0.336957				
Name: Survived, dtype: float64	Name: Survived, dtype: float64				
FamilySize	FamilySize				
0...0.303538	0...0.303538				
1...0.552795	1...0.552795				
2...0.578431	2...0.578431				
3...0.724138	3...0.724138				
4...0.200000	4...0.200000				
5...0.136364	5...0.136364				
6...0.333333	6...0.333333				
7...0.000000	7...0.000000				
10...0.000000	10...0.000000				
Name: Survived, dtype: float64	Name: Survived, dtype: float64				

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.

CODE:

```
import pandas as pd
import numpy as np

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 1. Get the number of survivors by gender
survivors = data[data['Survived'] == 1]
print(survivors['Sex'].value_counts())
# 2. Get the number of non-survivors by gender
non_survivors = data[data['Survived'] == 0]
print(non_survivors['Sex'].value_counts())
# 3. Get the number of survivors by embarked location
print(survivors['Embarked_S'].value_counts())
# 4. Get the number of non-survivors by embarked location
print(non_survivors['Embarked_S'].value_counts())
# 5. Calculate the percentage of children (Age < 18) who survived
children = data[data['Age'] < 18]
child_survival_rate = children['Survived'].mean()
print(child_survival_rate)
# 6. Calculate the percentage of adults (Age >= 18) who survived
adults = data[data['Age'] >= 18]
adult_survival_rate = adults['Survived'].mean()
print(adult_survival_rate)
# 7. Get the median age of survivors
print(survivors['Age'].median())
# 8. Get the median age of non-survivors
print(non_survivors['Age'].median())
print(survivors['Fare'].median())
print(non_survivors['Fare'].median())
```


Average time

0.016 s

16.00 ms



Maximum time

0.016 s

16.00 ms



✓ 1 out of 1 shown test case(s) passed

✓ Test case 1

16 ms



Debug



Expected output

Actual output

female...233

female...233

male...109

male...109

Name: Sex, dtype: int64

Name: Sex, dtype: int64

male...468

male...468

female...81

female...81

Name: Sex, dtype: int64

Name: Sex, dtype: int64

1...217

1...217

0...125

0...125

Name: Embarked_S, dtype: int64

Name: Embarked_S, dtype: int64

1...427

1...427

0...122

0...122

Name: Embarked_S, dtype: int64

Name: Embarked_S, dtype: int64

0.5398230088495575

0.5398230088495575

0.3810316139767055

0.3810316139767055

28.0

28.0

28.0

28.0

26.0

26.0